

Московский государственный технический университет
им. Н.Э. Баумана

Утверждаю:

Большаков С.А.

« » _____ 2026 г.

Курсовая работа по курсу «Системное программирование»
«Резидентная программа (TSR)»

Исходный текст программного продукта

(вид документа)

писчая бумага

(вид носителя)

23

(количество листов)

ИСПОЛНИТЕЛИ:

студент группы ИУ5-42Б

Воробьев Е. А., « » _____ 2026 г.

Москва — 2026

СОДЕРЖАНИЕ

1. Листинг программы <code>tsr.asm</code>	3
---	---

1. ЛИСТИНГ ПРОГРАММЫ TSR.ASM

Turbo Assembler Version 3.1
tsr.asm

05/13/26 14:04:29

Page 1

```
1          ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;          ;          ;
2          ; tsr.asm
3          ;
4          ; Сборка:
5          ; tasm.exe /l tsr.asm
6          ; tlink /t /x tsr.obj
7          ;
8          ; МГТУ им. Н.Э. Баумана, ИУ5-42Б, 2026 г.
9          ; Вариант №5: F8 - подпись (5 сек, верх), F9 - курсив 'З',
10         ;          F1 - русификация ЕЖЗИК, F2 - ограничить русские
буквы.
11         ; Воробьев Е. А.

12         ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;          ;          ;
13
14 0000      code segment      'code'
15          assume CS:code, DS:code
16          org      100h
17 0100      _start:
18
19 0100  E9 0709          jmp _initTSR ; на начало программы
20
21          ; данные
22          ;replaceWith          DB      ''
23          ;@ Вариант №5: ограничить ввод только русских букв
(строчные + прописные)
24 0103  A0 A1 A2 A3 A4 A5 F1+          ignoredChars          DB
+
25          A6 A7 A8 A9 AA AB AC+
'абвгдеёжзийклмнопрстуфхцчщщъыьэюяАБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧЩЩЪЫЬЭЮЯ' ; русские буквы
(ср866)
26          AD AE AF E0 E1 E2 E3+
27          E4 E5 E6 E7 E8 E9 EA+
28          EB EC ED EE EF 80 81+
29          82 83 84 85 F0 86 87+
30          88 89 8A 8B 8C 8D 8E+
31          8F 90 91 92 93 94 95+
32          96 97 98 99 9A 9B 9C+
33          9D 9E 9F
34          =0042          ignoredLength          equ      $-ignoredChars ;
длина строки      +
35          ignoredChars
36 0145  00          ignoreEnabled          DB      0          ; флаг
функции игнорирования      +
37          ввода
38          ;@ Вариант №5: русификация набора ЕЖЗИК (на ЙЦУКЕН
раскладке: Т -> Е, ; -> Ж, Р -> З, В+
39          -> И, R -> К)
40 0146  54 3B 50 42 52          translateFrom          DB      'Т;PBR' ;
латинские клавиши на местах      +
41          ЕЖЗИК
42 014B  85 86 87 88 8A          translateTo          DB
'ЕЖЗИК' ; русские буквы (ср866)
43          =0005          translateLength          equ      $-translateTo
; длина строки      +
44          translateFrom
45 0150  00          translateEnabled          DB      0          ;
флаг функции перевода
```

```

46
47 0151 00 signaturePrintingEnabled DB 0 ;
флаг функции вывода +
48 информации об авторе
49 0152 00 cursiveEnabled DB 0 ; флаг
перевода символа в курсив
50
51
52 ;@ Вариант №5: курсивная буква 'З' 8x16 (с наклоном вправо)
53 0153 00 cursiveSymbol DB 00000000b ; матрица курсивной
буквы 'З' (вместо обычного начертания)
54 0154 00 DB 00000000b
55 0155 3C DB 00111100b ; верхняя дуга
56 0156 66 DB 01100110b
57 0157 46 DB 01000110b

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 2
tsr.asm

```

58 0158 06 DB 00000110b
59 0159 0C DB 00001100b ; изгиб вправо к
середине
60 015A 3C DB 00111100b ; средняя
горизонталь
61 015B 0C DB 00001100b
62 015C 18 DB 00011000b
63 015D 30 DB 00110000b
64 015E 66 DB 01100110b
65 015F 66 DB 01100110b
66 0160 3C DB 00111100b ; нижняя дуга
67 0161 00 DB 00000000b
68 0162 00 DB 00000000b
69
70 0163 87 charToCursiveIndex DB 'З' ; символ для
замены (cp866)
71 0164 10*(FF) savedSymbol DB 16 dup(0FFh) ;
переменная для +
72 хранения старого символа
73
74 =00FF true equ
0FFh ; +
75 константа истинности
76 0174 ???? old_int9hOffset DW ? ;
адрес старого +
77 обработчика int 9h
78 0176 ???? old_int9hSegment DW ? ;
сегмент старого +
79 обработчика int 9h
80 0178 ???? old_int1ChOffset DW ? ;
адрес старого +
81 обработчика int 1Ch
82 017A ???? old_int1ChSegment DW ? ;
сегмент старого обработчика +
83 int 1Ch
84 017C ???? old_int2FhOffset DW ? ;
адрес старого +
85 обработчика int 2Fh
86 017E ???? old_int2FhSegment DW ? ;
сегмент старого обработчика +
87 int 2Fh
88
89 0180 00 unloadTSR DB 0 ; 1 -
выгрузить +
90 резидент
91 0181 00 notLoadTSR DB 0 ; 1 -
не загружать
92 0182 0000 counter DW 0
93 =0005 printDelay equ 5 ; задержка
перед выводом +

```

```

94          "подписи" в секундах
95 0184 0000          printPos          DW          0;
положение подписи на+
96          экране. 0 - верх, 1 - центр, 2 - низ
97
98          ;@ Вариант №5: ФИО, группа, номер варианта (длина первой
строки задаёт ширину рамки)
99 0186 B3 82 AE E0 AE A1 EC+          signatureLine1          DB          179,
'Воробьев Е. А.          +
100         A5 A2 20 85 2E 20 80+      ', 179
101         2E 20 20 20 20 20 20+
102         20 20 20 20 20 20 20+
103         20 20 20 20 20 20 20+
104         20 20 20 20 20 20 20+
105         20 20 20 20 20 20 20+
106         20 20 20 20 20 20 20+
107         B3
108         =0039          Line1_length          equ          $-
signatureLine1
109 01BF B3 88 93 35 2D 34 32+          signatureLine2          DB          179,
'ИУ5-42Б          +
110         81 20 20 20 20 20 20+      ', 179
111         20 20 20 20 20 20 20+
112         20 20 20 20 20 20 20+
113         20 20 20 20 20 20 20+
114         20 20 20 20 20 20 20+

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 3
tsr.asm

```

115         20 20 20 20 20 20 20+
116         20 20 20 20 20 20 20+
117         B3
118         =0039          Line2_length          equ          $-
signatureLine2
119 01F8 B3 82 A0 E0 A8 A0 AD+          signatureLine3          DB          179,
'Вариант №5          +
120         E2 20 FC 35 20 20 20+      ', 179
121         20 20 20 20 20 20 20+
122         20 20 20 20 20 20 20+
123         20 20 20 20 20 20 20+
124         20 20 20 20 20 20 20+
125         20 20 20 20 20 20 20+
126         20 20 20 20 20 20 20+
127         B3
128         =0039          Line3_length          equ          $-
signatureLine3
129 0231 3E 74 73 72 2E 63 6F+          helpMsg DB '>tsr.com [/?] ', 10, 13
130         6D 20 5B 2F 3F 5D 20+
131         0A 0D
132 0241 20 5B 2F 3F 5D 20 20+          DB ' [/?] - вывод данной
справки', 10, 13
133         20 2D 20 A2 EB A2 AE+
134         A4 20 A4 A0 AD AD AE+
135         A9 20 E1 AF E0 A0 A2+
136         AA A8 0A 0D
137 0261 20 20 54 53 52 2E 43+          DB ' TSR.COM - повторный
запуск выгружает резидент из +
138         4F 4D 20 2D 20 AF AE+ памяти', 10, 13
139         A2 E2 AE E0 AD EB A9+
140         20 A7 A0 AF E3 E1 AA+
141         20 A2 EB A3 E0 E3 A6+
142         A0 A5 E2 20 E0 A5 A7+
143         A8 A4 A5 AD E2 20 A8+
144         A7 20 AF A0 AC EF E2+
145         A8 0A 0D
146 029C 20 20 46 38 20 20 2D+          DB ' F8 - вывод ФИО и
группы по таймеру (5 сек) в +
147         20 A2 EB A2 AE A4 20+ верх экрана', 10, 13

```

```

148      94 88 8E 20 A8 20 A3+
149      E0 E3 AF AF EB 20 AF+
150      AE 20 E2 A0 A9 AC A5+
151      E0 E3 20 28 35 20 E1+
152      A5 AA 29 20 A2 20 A2+
153      A5 E0 E5 20 ED AA E0+
154      A0 AD A0 0A 0D
155 02D9 20 20 46 39 20 20 2D+      DB ' F9 - включение и
отключение курсивного вывода +
156      20 A2 AA AB EE E7 A5+      русского символа з', 10, 13
157      AD A8 A5 20 A8 20 AE+
158      E2 AA AB EE E7 A5 AD+
159      A8 A5 20 AA E3 E0 E1+
160      A8 A2 AD AE A3 AE 20+
161      A2 EB A2 AE A4 A0 20+
162      E0 E3 E1 E1 AA AE A3+
163      AE 20 E1 A8 AC A2 AE+
164      AB A0 20 87 0A 0D
165 031E 20 20 46 31 20 20 2D+      DB ' F1 - включение и
отключение частичной +
166      20 A2 AA AB EE E7 A5+      русификации клавиатуры (Т, ;, Р, В, R -> ЕЖЗИК)',
10, 13
167      AD A8 A5 20 A8 20 AE+
168      E2 AA AB EE E7 A5 AD+
169      A8 A5 20 E7 A0 E1 E2+
170      A8 E7 AD AE A9 20 E0+
171      E3 E1 A8 E4 A8 AA A0+

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 4
tsr.asm

```

172      E6 A8 A8 20 AA AB A0+
173      A2 A8 A0 E2 E3 E0 EB+
174      20 28 54 2C 20 3B 2C+
175      20 50 2C 20 42 2C 20+
176      52 20 2D 3E 20 85 86+
177      87 88 8A 29 0A 0D
178 0378 20 20 46 32 20 20 2D+      DB ' F2 - включение и
отключение режима ограничения +
179      20 A2 AA AB EE E7 A5+      ввода русских букв', 10, 13
180      AD A8 A5 20 A8 20 AE+
181      E2 AA AB EE E7 A5 AD+
182      A8 A5 20 E0 A5 A6 A8+
183      AC A0 20 AE A3 E0 A0+
184      AD A8 E7 A5 AD A8 EF+
185      20 A2 A2 AE A4 A0 20+
186      E0 E3 E1 E1 AA A8 E5+
187      20 A1 E3 AA A2 0A 0D
188
189      =018D      helpMsg_length      equ  $-helpMsg
190 03BE 8E E8 A8 A1 AA A0 20+      errorParamMsg      DB
'Ошибка параметров командной +
191      AF A0 E0 A0 AC A5 E2+      строки', 10, 13
192      E0 AE A2 20 AA AE AC+
193      AC A0 AD A4 AD AE A9+
194      20 E1 E2 E0 AE AA A8+
195      0A 0D
196      =0025      errorParamMsg_length      equ  $-errorParamMsg
197
198 03E3 DA 37*(C4) BF      tableTop      DB      218,
Line1_length-2 dup+
199      (196), 191
200      =0039      tableTop_length      equ  $-tableTop
201 041C C0 37*(C4) D9      tableBottom      DB      192,
Line1_length-2 dup (196), +
202      217
203      =0039      tableBottom_length      equ  $-tableBottom
204
205      ; сообщения

```

```

206 0455 90 A5 A7 A8 A4 A5 AD+ installedMsg DB 'Резидент
загружен!$'
207      E2 20 A7 A0 A3 E0 E3+
208      A6 A5 AD 21 24
209 0468 90 A5 A7 A8 A4 A5 AD+ alreadyInstalledMsg DB 'Резидент уже
загружен$'
210      E2 20 E3 A6 A5 20 A7+
211      A0 A3 E0 E3 A6 A5 AD+
212      24
213 047E 8D A5 A4 AE E1 E2 A0+ noMemMsg DB
'Недостаточно памяти$'
214      E2 AE E7 AD AE 20 AF+
215      A0 AC EF E2 A8 24
216 0492 8D A5 20 E3 A4 A0 AB+ notInstalledMsg DB 'He
удалось загрузить резидент$'
217      AE E1 EC 20 A7 A0 A3+
218      E0 E3 A7 A8 E2 EC 20+
219      E0 A5 A7 A8 A4 A5 AD+
220      E2 24
221
222 04B0 90 A5 A7 A8 A4 A5 AD+ removedMsg DB 'Резидент
выгружен'
223      E2 20 A2 EB A3 E0 E3+
224      A6 A5 AD
225      =0011 removedMsg_length equ $-removedMsg
226
227 04C1 8D A5 20 E3 A4 A0 AB+ noRemoveMsg DB 'He
удалось выгрузить резидент'
228      AE E1 EC 20 A2 EB A3+

Turbo Assembler Version 3.1      05/13/26 14:04:29      Page 5
tsr.asm

```

```

229      E0 E3 A7 A8 E2 EC 20+
230      E0 A5 A7 A8 A4 A5 AD+
231      E2
232      =001D noRemoveMsg_length equ $-noRemoveMsg
233
234      ;@ Вариант №5: индикатор активных функций в правом нижнем
углу
235 04DE 46 38 f1_txt DB 'F8' ; вывод
подписи
236 04E0 46 39 f2_txt DB 'F9' ; курсив 'З'
237 04E2 46 31 f3_txt DB 'F1' ;
русификация ЕЖЗИК
238 04E4 46 32 f4_txt DB 'F2' ;
ограничение русских букв
239      =0002 fx_length equ $-f4_txt
240
241 04E6      changeFx proc
242 04E6 50      push AX
243 04E7 53      push BX
244 04E8 51      push CX
245 04E9 52      push DX
246 04EA 55      push BP
247 04EB 06      push ES
248 04EC 33 DB      xor BX, BX
249
250 04EE B4 03      mov AH, 03h
251 04F0 CD 10      int 10h
252 04F2 52      push DX
253
254 04F3 0E      push CS
255 04F4 07      pop ES
256
257 04F5      _checkF5:
258 04F5 BD 04DEr      lea BP, f1_txt
259 04F8 B9 0002      mov CX, fx_length
260 04FB B7 00      mov BH, 0

```

261	04FD	B6 00	mov DH, 0
262	04FF	B2 4E	mov DL, 78
263	0501	B8 1301	mov AX, 1301h
264			
265	0504	80 3E 0151r FF	cmp signaturePrintingEnabled, true
266	0509	74 07	je _greenF5
267			
268	050B		_redF5:
269	050B	B3 4F	mov BL, 01001111b ; red
270	050D	CD 10	int 10h
271	050F	EB 08 90	jmp _checkF6
272			
273	0512		_greenF5:
274	0512	BD 04DEr	lea BP, f1_txt
275	0515	B3 2F	mov BL, 00101111b ; green
276	0517	CD 10	int 10h
277			
278	0519		_checkF6:
279	0519	BD 04E0r	lea BP, f2_txt
280	051C	B9 0002	mov CX, fx_length
281	051F	B7 00	mov BH, 0
282	0521	B6 01	mov DH, 1
283	0523	B2 4E	mov DL, 78
284	0525	B8 1301	mov AX, 1301h
285			

Turbo Assembler Version 3.1
tsr.asm

05/13/26 14:04:29

Page 6

286	0528	80 3E 0152r FF	cmp cursiveEnabled, true
287	052D	74 49	je _greenF8
288			
289	052F		_redF6:
290	052F	B3 4F	mov BL, 01001111b ; red
291	0531	CD 10	int 10h
292	0533	EB 05 90	jmp _checkF7
293			
294	0536		_greenF6:
295	0536	B3 2F	mov BL, 00101111b ; green
296	0538	CD 10	int 10h
297			
298	053A		_checkF7:
299	053A	BD 04E2r	lea BP, f3_txt
300	053D	B9 0002	mov CX, fx_length
301	0540	B7 00	mov BH, 0
302	0542	B6 02	mov DH, 2
303	0544	B2 4E	mov DL, 78
304	0546	B8 1301	mov AX, 1301h
305			
306	0549	80 3E 0150r FF	cmp translateEnabled, true
307	054E	74 07	je _greenF7
308			
309	0550		_redF7:
310	0550	B3 4F	mov BL, 01001111b ; red
311	0552	CD 10	int 10h
312	0554	EB 05 90	jmp _checkF8
313			
314	0557		_greenF7:
315	0557	B3 2F	mov BL, 00101111b ; green
316	0559	CD 10	int 10h
317			
318	055B		_checkF8:
319	055B	BD 04E4r	lea BP, f4_txt
320	055E	B9 0002	mov CX, fx_length
321	0561	B7 00	mov BH, 0
322	0563	B6 03	mov DH, 3
323	0565	B2 4E	mov DL, 78
324	0567	B8 1301	mov AX, 1301h
325			

```

326 056A 80 3E 0145r FF          cmp ignoreEnabled, true
327 056F 74 07          je _greenF8
328
329 0571          _redF8:
330 0571 B3 4F          mov BL, 01001111b ; red
331 0573 CD 10          int 10h
332 0575 EB 05 90          jmp _outFx
333
334 0578          _greenF8:
335 0578 B3 2F          mov BL, 00101111b ; green
336 057A CD 10          int 10h
337
338 057C          _outFx:
339 057C 5A          pop DX
340 057D B4 02          mov AH, 02h
341 057F CD 10          int 10h
342

```

Turbo Assembler Version 3.1
tsr.asm

05/13/26 14:04:29

Page 7

```

343 0581 07          pop ES
344 0582 5D          pop BP
345 0583 5A          pop DX
346 0584 59          pop CX
347 0585 5B          pop BX
348 0586 58          pop AX
349 0587 C3          ret
350 0588          changeFx endp
351
352          ;новый обработчик
353 0588          new_int9h proc far
354          ; сохраняем значения всех, изменяемых регистров в
стэке
355 0588 56          push SI
356 0589 50          push AX
357 058A 53          push BX
358 058B 51          push CX
359 058C 52          push DX
360 058D 06          push ES
361 058E 1E          push DS
362          ; синхронизируем CS и DS
363 058F 0E          push CS
364 0590 1F          pop DS
365
366 0591 B8 0040          mov AX, 40h ; 40h-сегмент, где хранятся
флаги сост-я клавиатуры, кольцо. +
367          буфер ввода
368 0594 8E C0          mov ES, AX
369 0596 E4 60          in AL, 60h ; записываем в AL скан-код
нажатой клавиши
370
371
372
373
374          ;@ Вариант №5: проверка функциональных клавиш F8, F9,
F1, F2
375          ;@ Скан-коды: F1=3Bh F2=3Ch F8=42h F9=43h
376          ;@ После sub AL,58: F1->1 F2->2 F8->8 F9->9
377
378          ;проверка F8/F9/F1/F2
379 0598          _test_Fx:
380 0598 2C 3A          sub AL, 58 ; в AL теперь номер функциональной
клавиши (F1=1..F10=10)
381 059A          _F8:
382 059A 3C 08          cmp AL, 8 ; F8 - подпись
383 059C 75 0A          jne _F9
384 059E F6 16 0151r          not signaturePrintingEnabled
385 05A2 E8 FF41          call changeFx

```

```

386 05A5 EB 2E 90 jmp _translate_or_ignore
387 05A8 _F9:
388 05A8 3C 09 cmp AL, 9 ; F9 - курсив '3'
389 05AA 75 0D jne _F1
390 05AC F6 16 0152r not cursiveEnabled
391 05B0 E8 FF33 call changeFx
392 05B3 E8 01EF call setCursive ; перевод символа в
курсив и обратно в зависимости от +
393 флага cursiveEnabled
394 05B6 EB 1D 90 jmp _translate_or_ignore
395 05B9 _F1:
396 05B9 3C 01 cmp AL, 1 ; F1 - русификация ЕЖЗИК
397 05BB 75 0A jne _F2
398 05BD F6 16 0150r not translateEnabled
399 05C1 E8 FF22 call changeFx

```

Turbo Assembler Version 3.1
tsr.asm

05/13/26 14:04:29

Page 8

```

400 05C4 EB 0F 90 jmp _translate_or_ignore
401 05C7 _F2:
402 05C7 3C 02 cmp AL, 2 ; F2 - ограничение ввода русских
букв
403 05C9 75 0A jne _translate_or_ignore
404 05CB F6 16 0145r not ignoreEnabled
405 05CF E8 FF14 call changeFx
406 05D2 EB 01 90 jmp _translate_or_ignore
407
408 ;игнорирование и перевод
409 05D5 _translate_or_ignore:
410
411 05D5 9C pushf
412 05D6 2E: FF 1E 0174r call dword ptr CS:[old_int9hOffset] ;
вызываем стандартный обработчик прерывания
413 05DB B8 0040 mov AX, 40h ; 40h-сегмент, где
хранятся флаги сост-я клавиш, колец. +
414 буфер ввода
415 05DE 8E C0 mov ES, AX
416 05E0 26: 8B 1E 001C mov BX, ES:[1Ch] ; адрес хвоста
417 05E5 4B dec BX ; сместимся назад к последнему
418 05E6 4B dec BX ; введённому символу
419 05E7 83 FB 1E cmp BX, 1Eh ; не вышли ли мы за пределы
буфера?
420 05EA 73 03 jae _go
421 05EC BB 003C mov BX, 3Ch ; хвост вышел за пределы
буфера, значит последний введённый +
422 символ
423 ; находится в конце буфера
424
425 05EF _go:
426 05EF 26: 8B 17 mov DX, ES:[BX] ; в DX 0 введённый символ
427 ;включен ли режим блокировки ввода?
428 05F2 80 3E 0145r FF cmp ignoreEnabled, true
429 05F7 75 1A jne _check_translate
430
431 ; да, включен
432 05F9 BE 0000 mov SI, 0
433 05FC B9 0042 mov CX, ignoredLength ;кол-во игнорируемых
символов
434
435 ; проверяем, присутствует ли текущий символ в списке
игнорируемых
436 05FF _check_ignored:
437 05FF 3A 94 0103r cmp DL, ignoredChars[SI]
438 0603 74 06 je _block
439 0605 46 inc SI
440 0606 E2 F7 loop _check_ignored
441 0608 EB 09 90 jmp _check_translate
442

```

```

443                                     ; блокируем
444 060B                                _block:
445 060B 26: 89 1E 001C                mov ES:[1Ch], BX ;блокировка ввода
символа
446                                     ;@ если по варианту нужно не блокировать ввод
символа,
447                                     ;@ а заменять одни символы другими,
448                                     ;@ замените строку выше строкой
449                                     ;mov ES:[BX], AX
450                                     ;@ на месте AX может быть '*' для замены всех
символов множества ignoredChars +
451                                     на звездочки
452                                     ;@ или, для перевода одних символов в другие -
завести массив
453                                     ;@ replaceWith DB '...', где перечислить символы, на
которые пойдёт замена
454                                     ;@ и раскомментировать строки ниже:
455                                     ;xor AX, AX
456                                     ;mov AL, replaceWith[SI]

Turbo Assembler Version 3.1          05/13/26 14:04:29          Page 9
tsr.asm

```

```

457                                     ;mov ES:[BX], AX      ; замена символа
458 0610 EB 23 90                        jmp _quit
459
460 0613                                _check_translate:
461                                     ; включен ли режим перевода?
462 0613 80 3E 0150r FF                cmp translateEnabled, true
463 0618 75 1B                          jne _quit
464
465                                     ; да, включен
466 061A BE 0000                        mov SI, 0
467 061D B9 0005                        mov CX, translateLength ; кол-во символов для
перевода
468                                     ; проверяем, присутствует ли текущий символ в списке
для перевода
469 0620                                _check_translate_loop:
470 0620 3A 94 0146r                    cmp DL, translateFrom[SI]
471 0624 74 06                          je _translate
472 0626 46                            inc SI
473 0627 E2 F7                          loop _check_translate_loop
474 0629 EB 0A 90                        jmp _quit
475
476                                     ; переводим
477 062C                                _translate:
478 062C 33 C0                          xor AX, AX
479 062E 8A 84 014Br                    mov AL, translateTo[SI]
480 0632 26: 89 07                      mov ES:[BX], AX ; замена символа
481
482 0635                                _quit:
483                                     ; восстанавливаем все регистры
484 0635 1F                              pop     DS
485 0636 07                              pop     ES
486 0637 5A                              pop     DX
487 0638 59                              pop     CX
488 0639 5B                              pop     BX
489 063A 58                              pop     AX
490 063B 5E                              pop     SI
491 063C CF                              iret
492 063D                                new_int9h endp
493
494                                     ;=== Обработчик прерывания int 1Ch ===;
495                                     ;=== Вызывается каждые 55 мс ===;
496 063D                                new_int1Ch proc far
497 063D 50                              push AX
498 063E 0E                              push CS
499 063F 1F                              pop     DS
500

```

```

501 0640 9C                      pushf
502 0641 2E: FF 1E 0178r          call dword ptr CS:[old_int1ChOffset]
503
504 0646 80 3E 0151r FF          cmp signaturePrintingEnabled, true ; если
нажата управляющая клавиша (в данном случае +
505                      F6)
506 064B 75 1C                      jne _notToPrint
507
508 064D 83 3E 0182r 5B          cmp counter, printDelay*1000/55 + 1 ;
если кол-во "тактов" эквивалентно +
509                      %printDelay% секундам
510 0652 74 03                      je _letsPrint
511
512 0654 EB 0E 90                      jmp _dontPrint
513

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 10
tsr.asm

```

514 0657                      _letsPrint:
515 0657 F6 16 0151r          not signaturePrintingEnabled
516 065B C7 06 0182r 0000          mov counter, 0
517 0661 E8 0094                      call printSignature
518
519 0664                      _dontPrint:
520 0664 83 06 0182r 01          add counter, 1
521
522 0669                      _notToPrint:
523
524 0669 58                      pop AX
525
526 066A CF                      iret
527 066B                      new_int1Ch endp
528
529                      ;=== Обработчик прерывания int 2Fh ===;
530                      ;=== Служит для:
531                      ;=== 1) проверки факта присутствия TSR в памяти (при
АН=0FFh, AL=0)
532                      ;=== будет возвращён АН='i' в случае, если TSR уже
загружен
533                      ;=== 2) выгрузки TSR из памяти (при АН=0FFh, AL=1)
534                      ;===
535 066B                      new_int2Fh proc
536 066B 80 FC FF          cmp AH, 0FFh ;наша функция?
537 066E 75 0B          jne _2Fh_std ;нет - на старый обработчик
538 0670 3C 00          cmp AL, 0 ;подфункция проверки, загружен ли
резидент в память?
539 0672 74 0C          je _already_installed
540 0674 3C 01          cmp AL, 1 ;подфункция выгрузки из памяти?
541 0676 74 0B          je _uninstall
542 0678 EB 01 90          jmp _2Fh_std ;нет - на старый
обработчик
543
544 067B                      _2Fh_std:
545 067B 2E: FF 2E 017Cr          jmp dword ptr CS:
[old_int2FhOffset] ;вызов старого обработчика
546
547 0680                      _already_installed:
548 0680 B4 69          mov AH, 'i' ;вернём 'i', если резидент
загружен в память
549 0682 CF                      iret
550
551 0683                      _uninstall:
552 0683 1E          push DS
553 0684 06          push ES
554 0685 52          push DX
555 0686 53          push BX
556
557 0687 33 DB          xor BX, BX

```

```

558
559                                ; CS = ES, для доступа к переменным
560 0689 0E                        push CS
561 068A 07                        pop ES
562
563 068B B8 2509                   mov     AX, 2509h
564 068E 26: 8B 16 0174r           mov     DX, ES:old_int9hOffset      ; возвращаем
вектор прерывания
565 0693 26: 8E 1E 0176r           mov     DS, ES:old_int9hSegment      ; на место
566 0698 CD 21                     int     21h
567
568 069A B8 251C                   mov     AX, 251Ch
569 069D 26: 8B 16 0178r           mov     DX, ES:old_int1ChOffset      ; возвращаем
вектор прерывания
570 06A2 26: 8E 1E 017Ar           mov     DS, ES:old_int1ChSegment      ; на место

Turbo Assembler Version 3.1      05/13/26 14:04:29      Page 11
tsr.asm

571 06A7 CD 21                     int     21h
572
573 06A9 B8 252F                   mov     AX, 252Fh
574 06AC 26: 8B 16 017Cr           mov     DX, ES:old_int2FhOffset      ; возвращаем
вектор прерывания
575 06B1 26: 8E 1E 017Er           mov     DS, ES:old_int2FhSegment      ; на место
576 06B6 CD 21                     int     21h
577
578 06B8 2E: 8E 06 002C            mov     ES, CS:2Ch      ; загрузим в ES адрес
окружения
579 06BD B4 49                     mov     AH, 49h      ; выгрузим из памяти
окружение
580 06BF CD 21                     int     21h
581 06C1 72 0B                     jc      _notRemove
582
583 06C3 0E                        push     CS
584 06C4 07                        pop      ES      ; в ES - адрес резидентной программы
585 06C5 B4 49                     mov     AH, 49h      ; выгрузим из памяти резидент
586 06C7 CD 21                     int     21h
587 06C9 72 03                     jc      _notRemove
588 06CB EB 15 90                   jmp     _unloaded
589
590 06CE                           _notRemove: ; не удалось выполнить выгрузку
591                                ; вывод сообщения о неудачной выгрузке
592 06CE B4 03                     mov     AH, 03h      ; получаем позицию
курсора
593 06D0 CD 10                     int     10h
594 06D2 BD 04C1r                  lea     BP, noRemoveMsg
595 06D5 B9 001D                   mov     CX, noRemoveMsg_length
596 06D8 B3 07                     mov     BL, 0111b
597 06DA B8 1301                   mov     AX, 1301h
598 06DD CD 10                     int     10h
599 06DF EB 12 90                   jmp     _2Fh_exit
600
601 06E2                           _unloaded: ; выгрузка прошла успешно
602                                ; вывод сообщения об удачной выгрузке
603 06E2 B4 03                     mov     AH, 03h      ; получаем позицию
курсора
604 06E4 CD 10                     int     10h
605 06E6 BD 04B0r                  lea     BP, removedMsg
606 06E9 B9 0011                   mov     CX, removedMsg_length
607 06EC B3 07                     mov     BL, 0111b
608 06EE B8 1301                   mov     AX, 1301h
609 06F1 CD 10                     int     10h
610
611 06F3                           _2Fh_exit:
612 06F3 5B                        pop     BX
613 06F4 5A                        pop     DX
614 06F5 07                        pop     ES
615 06F6 1F                        pop     DS

```

```

616 06F7 CF          ired
617 06F8              new_int2Fh endp
618
619                  ;=== Процедура вывода подписи (ФИО, группа)
620                  ;=== Настраивается значениями переменных в начале исходника
621                  ;===
622 06F8              printSignature proc
623 06F8 50            push AX
624 06F9 52            push DX
625 06FA 51            push CX
626 06FB 53            push BX
627 06FC 06            push ES

Turbo Assembler Version 3.1      05/13/26 14:04:29      Page 12
tsr.asm

628 06FD 54            push SP
629 06FE 55            push BP
630 06FF 56            push SI
631 0700 57            push DI
632
633 0701 33 C0          xor AX, AX
634 0703 33 DB          xor BX, BX
635 0705 33 D2          xor DX, DX
636
637 0707 B4 03          mov AH, 03h                      ;чтение текущей
позиции курсора
638 0709 CD 10          int 10h
639 070B 52            push DX                          ;помещаем
информацию о      +
640                  положении курсора в стек
641
642 070C 83 3E 0184r 00      cmp printPos, 0
643 0711 74 0E            je _printTop
644
645 0713 83 3E 0184r 01      cmp printPos, 1
646 0718 74 0E            je _printCenter
647
648 071A 83 3E 0184r 02      cmp printPos, 2
649 071F 74 0E            je _printBottom
650
651                  ;все числа подобраны на глаз...
652 0721                _printTop:
653 0721 B6 00            mov DH, 0
654 0723 B2 0F            mov DL, 15
655 0725 EB 0F 90          jmp _actualPrint
656
657 0728                _printCenter:
658 0728 B6 09            mov DH, 9
659 072A B2 0F            mov DL, 15
660 072C EB 08 90          jmp _actualPrint
661
662 072F                _printBottom:
663 072F B6 13            mov DH, 19
664 0731 B2 0F            mov DL, 15
665 0733 EB 01 90          jmp _actualPrint
666
667 0736                _actualPrint:
668 0736 B4 0F            mov AH, 0Fh                      ;чтение текущего
видеорежима. в+
669                  BH - текущая страница
670 0738 CD 10          int 10h
671
672 073A 0E            push CS
673 073B 07            pop ES                          ;указываем ES на CS
674
675                  ;вывод 'верхушки' таблицы
676 073C 52            push DX
677 073D BD 03E3r        lea BP, tableTop                      ;помещаем

```

```

в BP указатель на      +
678                                выводимую строку
679 0740 B9 0039                                mov CX, tableTop_length                ;в CX - длина
строки
680 0743 B3 07                                mov BL, 0111b                ;цвет выводимого
текста ref:      +
681                                http://en.wikipedia.org/wiki/BIOS_color_attributes
682 0745 B8 1301                                mov AX, 1301h                ;AH=13h -
номер ф-ии, AL=01h - +
683                                курсор перемещается при выводе каждого из символов строки
684 0748 CD 10                                int 10h

Turbo Assembler Version 3.1                05/13/26 14:04:29                Page 13
tsr.asm

```

```

685 074A 5A                                pop DX
686 074B FE C6                                inc DH
687
688
689                                ;вывод первой линии
690 074D 52                                push DX
691 074E BD 0186r                            lea BP, signatureLine1
692 0751 B9 0039                                mov CX, Line1_length
693 0754 B3 07                                mov BL, 0111b
694 0756 B8 1301                                mov AX, 1301h
695 0759 CD 10                                int 10h
696 075B 5A                                pop DX
697 075C FE C6                                inc DH
698
699                                ;вывод второй линии
700 075E 52                                push DX
701 075F BD 01BFr                            lea BP, signatureLine2
702 0762 B9 0039                                mov CX, Line2_length
703 0765 B3 07                                mov BL, 0111b
704 0767 B8 1301                                mov AX, 1301h
705 076A CD 10                                int 10h
706 076C 5A                                pop DX
707 076D FE C6                                inc DH
708
709                                ;вывод третьей линии
710 076F 52                                push DX
711 0770 BD 01F8r                            lea BP, signatureLine3
712 0773 B9 0039                                mov CX, Line3_length
713 0776 B3 07                                mov BL, 0111b
714 0778 B8 1301                                mov AX, 1301h
715 077B CD 10                                int 10h
716 077D 5A                                pop DX
717 077E FE C6                                inc DH
718
719                                ;вывод 'низа' таблицы
720 0780 52                                push DX
721 0781 BD 041Cr                            lea BP, tableBottom
722 0784 B9 0039                                mov CX, tableBottom_length
723 0787 B3 07                                mov BL, 0111b
724 0789 B8 1301                                mov AX, 1301h
725 078C CD 10                                int 10h
726 078E 5A                                pop DX
727 078F FE C6                                inc DH
728
729 0791 33 DB                                xor BX, BX
730 0793 5A                                pop DX                                ;восстанавливаем из
стека      +
731                                прежнее положение курсора
732 0794 B4 02                                mov AH, 02h                                ;меняем положение
курсора на      +
733                                первоначальное
734 0796 CD 10                                int 10h
735 0798 E8 FD4B                            call changeFx
736

```

```

737 079B 5F          pop DI
738 079C 5E          pop SI
739 079D 5D          pop BP
740 079E 5C          pop SP
741 079F 07          pop ES

```

Turbo Assembler Version 3.1
tsr.asm

05/13/26 14:04:29

Page 14

```

742 07A0 5B          pop BX
743 07A1 59          pop CX
744 07A2 5A          pop DX
745 07A3 58          pop AX
746
747 07A4 C3          ret
748 07A5          printSignature endp
749
750          ;=== Функция, которая в зависимости от флага cursiveEnabled
меняет начертание символа с курсива+
751          на обычное и наоборот
752          ;=== Сама смена происходит в процедуре changeFont, а здесь
подготавливаются данные
753 07A5          setCursive proc
754 07A5 06          push ES ; сохраняем регистры
755 07A6 50          push AX
756 07A7 0E          push CS
757 07A8 07          pop ES
758
759 07A9 80 3E 0152r FF      cmp cursiveEnabled, true
760 07AE 75 30          jne _restoreSymbol
761          ; если флаг равен true, выполняем замену символа на
курсивный вариант,
762          ; предварительно сохраняя старый символ в savedSymbol
763
764 07B0 E8 004C          call saveFont
765 07B3 8A 0E 0163r      mov CL, charToCursiveIndex
766 07B7          _shiftTtable:
767          ; мы получаем в BP таблицу всех символов. адрес указывает
на символ 0
768          ; поэтому нужно совершить сдвиг 16*X - где X - код
символа
769 07B7 83 C5 10          add BP, 16
770 07BA E2 FB          loop _shiftTable
771
772          ; при savefont смещается регистр ES
773          ; поэтому приходится делать такие махинации, чтобы
774          ; записать полученный элемент в savedSymbol
775          ; swap(ES, DS) и сохранение старого значения DS
776 07BC 1E          push DS
777 07BD 58          pop AX
778 07BE 06          push ES
779 07BF 1F          pop DS
780 07C0 50          push AX
781 07C1 07          pop ES
782 07C2 50          push AX
783
784 07C3 8B F5          mov SI, BP
785 07C5 BF 0164r      lea DI, savedSymbol
786          ; сохраняем в переменную savedSymbol
787          ; таблицу нужного символа
788 07C8 B9 0010          mov CX, 16
789          ; movsb из DS:SI в ES:DI
790 07CB F3> A4          rep movsb
791          ; исходные позиции сегментов возвращены
792 07CD 1F          pop DS ; восстановление DS
793
794          ; заменим написание символа на курсив
795 07CE B9 0001          mov CX, 1
796 07D1 B6 00          mov DH, 0

```

```

797 07D3 8A 16 0163r      mov DL, charToCursiveIndex
798 07D7 BD 0153r         lea BP, cursiveSymbol

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 15
tsr.asm

```

799 07DA E8 0015          call changeFont
800 07DD EB 10 90          jmp _exitSetCursive
801
802 07E0                  _restoreSymbol:
803                      ; если флаг равен 0, выполняем замену курсивного символа
на старый вариант
804
805 07E0 B9 0001          mov CX, 1
806 07E3 B6 00          mov DH, 0
807 07E5 8A 16 0163r      mov DL, charToCursiveIndex
808 07E9 BD 0164r         lea bp, savedSymbol
809 07EC E8 0003          call changeFont
810
811 07EF                  _exitSetCursive:
812 07EF 58              pop AX
813 07F0 07              pop ES
814 07F1 C3              ret
815 07F2                  setCursive endp
816
817                      ;=== Функция смены начертания символа (курсив/нормальное)
818                      ;===
819                      ; *** входные данные
820                      ; DL = номер символа для замены
821                      ; CX = Кол-во символов заменяемых изображений символов
822                      ; (начиная с символа указанного в DX)
823                      ; ES:bp = адрес таблицы
824                      ;
825                      ; *** описание работы процедуры
826                      ; Происходит вызов int 10h (видеосервис)
827                      ; с функцией AH = 11h (функции знакогенератора)
828                      ; Параметр AL = 0 сообщает, что будет заменено изображение
829                      ; символа для текущего шрифта
830                      ; В случаях, когда AL = 1 или 2, будет заменено изображение
831                      ; только для определенного шрифта (8x14 и 8x8 соответственно)
832                      ; Параметр BH = 0Eh сообщает, что на определение каждого
изображения символа
833                      ; расходуется по 14 байт (режим 8x14 бит как раз 14 байт)
834                      ; Параметр BL = 0 - блок шрифта для загрузки (от 0 до 4)
835                      ;
836                      ; *** результат
837                      ; изображение указанного(ых) символа(ов) будет заменено
838                      ; на предложенное пользователем.
839                      ; Изменению подвергнутся все символы, находящиеся на экране,
840                      ; то есть если изображение заменено, старый вариант нигде уже
не проявится
841
842 07F2                  changeFont proc
843 07F2 50              push AX
844 07F3 53              push BX
845 07F4 B8 1100          mov AX, 1100h
846 07F7 BB 1000          mov BX, 1000h
847 07FA CD 10              int 10h
848 07FC 58              pop AX
849 07FD 5B              pop BX
850 07FE C3              ret
851 07FF                  changeFont endp
852
853                      ;=== Функция сохранения нормального начертания символа
854                      ;===
855                      ; *** входные данные

```

Turbo Assembler Version 3.1 05/13/26 14:04:29 Page 16
tsr.asm

```

856             ; BH - тип возвращаемой символьной таблицы
857             ; 0 - таблица из int 1fh
858             ; 1 - таблица из int 44h
859             ; 2-5 - таблица из 8x14, 8x8, 8x8 (top), 9x14
860             ; 6 - 8x16
861             ;
862             ; *** описание работы процедуры
863             ; Происходит вызов int 10h (видеосервис)
864             ; с функцией AH = 11h (функции знакогенератора)
865             ; Параметр AL = 30 - подфункция получения информации о EGA
866             ;
867             ; *** результат
868             ; в ES:BP находится таблица символов (полная)
869             ; в CX находится байт на символ
870             ; в DL количество экранных строк
871             ; ВАЖНО! Происходит сдвиг регистра ES
872             ; ( ES становится равным C000h )
873
874 07FF             saveFont proc
875 07FF 50             push AX
876 0800 53             push BX
877 0801 B8 1130         mov AX, 1130h
878 0804 BB 0600         mov BX, 0600h
879 0807 CD 10             int 10h
880 0809 58             pop AX
881 080A 5B             pop BX
882 080B C3             ret
883 080C             saveFont endp
884
885
886             ;=== Отсюда начинается выполнение основной части программы
===;
887             ;===
888 080C             _initTSR: ; старт резидента
889 080C B4 03             mov AH, 03h
890 080E CD 10             int 10h
891 0810 52             push DX
892 0811 B4 00             mov AH,00h ; установка видеорежима (83h текст 80x25
16/8 CGA,EGA b800 Comp,RGB, +
893             Enhanced), без очистки экрана
894 0813 B0 83             mov AL,83h
895 0815 CD 10             int 10h
896 0817 5A             pop DX
897 0818 B4 02             mov AH, 02h
898 081A CD 10             int 10h
899
900
901 081C E8 0095         call commandParamsParser
902 081F B8 3509         mov AX,3509h ; получить в ES:BX вектор
09
903 0822 CD 21             int 21h ; прерывания
904
905             ;@ === Удаление резидента из памяти ===
906             ;@ Если по варианту необходимо выгружать резидент по
повторному запуску приложений,
907             ;@ нужно закомментировать следующие 3 строки, а также
908             ;@ содержимое метки _finishTSR ф-ии commandParamsParser,
но не саму метку!
909             ;cmp unloadTSR, true
910             ;je _removingOnParameter
911             ;jmp _notRemovingNow
912

```

```

913 0824                _removingOnParameter:
914 0824  B4 FF          mov AH, 0FFh
915 0826  B0 00          mov AL, 0
916 0828  CD 2F          int 2Fh
917 082A  80 FC 69       cmp AH, 'i' ; проверка того, загружена ли
уже программа
918 082D  74 69         je _remove
919                                ;mov AH, 09h                ;@ для выгрузки резидента
по повторному+
920                                запуску закомментировать эту строку
921                                ;lea DX, notInstalledMsg        ;@ для выгрузки
резидента по повторному запуску+
922                                закомментировать эту строку
923                                ;int 21h                        ;@ для выгрузки
резидента по      +
924                                повторному запуску закомментировать эту строку
925                                ;int 20h                        ;@ для выгрузки
резидента по      +
926                                повторному запуску закомментировать эту строку
927
928 082F                _notRemovingNow:
929
930 082F  80 3E 0181r FF   cmp notLoadTSR, true                ; если была
выведена справка
931 0834  74 03          je _exit_tmp                        ; просто выходим
932
933                                ;@ Если по варианту необходимо выгружать резидент по
повторному запуску, то      +
934                                комментируем 5 строк ниже
935                                ;@ если необходимо выгружать по параметру командной
строки, то оставляем их
936                                ;mov AH, 0FFh
937                                ;mov AL, 0
938                                ;int 2Fh
939                                ;cmp AH, 'i' ; проверка того, загружена ли уже программа
940                                ;je _alreadyInstalled
941
942 0836  EB 04 90       jmp _tmp
943
944 0839                _exit_tmp:
945 0839  EB 77 90       jmp _exit
946
947 083C                _tmp:
948 083C  06            push ES
949 083D  A1 002C       mov AX, DS:[2Ch]                ; psp
950 0840  8E C0       mov ES, AX
951 0842  B4 49       mov AH, 49h                ; хватит памяти чтоб остаться
952 0844  CD 21       int 21h                ; резидентом?
953 0846  07         pop ES
954 0847  72 62       jc _notMem                ; не хватило - выходим
955
956                                ;== int 09h ==;
957
958 0849  2E: 89 1E 0174r   mov     word ptr CS:old_int9hOffset, BX
959 084E  2E: 8C 06 0176r   mov     word ptr CS:old_int9hSegment, ES
960 0853  B8 2509         mov AX, 2509h                ; установим вектор на 09
961 0856  BA 0588r       mov DX, offset new_int9h        ; прерывание
962 0859  CD 21         int 21h
963
964                                ;== int 1Ch ==;
965 085B  B8 351C         mov AX, 351Ch                ; получить в ES:BX вектор
1C
966 085E  CD 21         int 21h                ; прерывания
967 0860  2E: 89 1E 0178r   mov     word ptr CS:old_int1ChOffset, BX
968 0865  2E: 8C 06 017Ar   mov     word ptr CS:old_int1ChSegment, ES
969 086A  B8 251C         mov AX, 251Ch                ; установим вектор на 1C

```

```

970 086D BA 063Dr          mov DX, offset new_int1Ch          ; прерывание
971 0870 CD 21             int 21h
972
973                         ;== int 2Fh ==;
974 0872 B8 352F           mov AX, 352Fh                      ; получить в ES:BX вектор
1C
975 0875 CD 21             int 21h                          ; прерывания
976 0877 2E: 89 1E 017Cr    mov word ptr CS:old_int2FhOffset, BX
977 087C 2E: 8C 06 017Er    mov word ptr CS:old_int2FhSegment, ES
978 0881 B8 252F           mov AX, 252Fh                    ; установим вектор на 2F
979 0884 BA 066Br          mov DX, offset new_int2Fh          ; прерывание
980 0887 CD 21             int 21h
981
982 0889 E8 FC5A           call changeFx
983 088C BA 0455r          mov DX, offset installedMsg        ; выводим что все
OK
984 088F B4 09            mov AH, 9
985 0891 CD 21             int 21h
986 0893 BA 080Cr          mov DX, offset _initTSR            ; остаемся в памяти
резидентом
987 0896 CD 27             int 27h                          ; и выходим
988                         ; конец основной программы
989 0898                   _remove: ; выгрузка программы из памяти
990 0898 B4 FF             mov AH, 0FFh
991 089A B0 01             mov AL, 1
992 089C CD 2F             int 2Fh
993 089E EB 12 90          jmp _exit
994 08A1                   _alreadyInstalled:
995 08A1 B4 09             mov AH, 09h
996 08A3 BA 0468r          lea DX, alreadyInstalledMsg
997 08A6 CD 21             int 21h
998 08A8 EB 08 90          jmp _exit
999 08AB                   _notMem: ; не хватает памяти, чтобы
остаться резидентом
1000 08AB BA 047Er          mov DX, offset noMemMsg
1001 08AE B4 09            mov AH, 9
1002 08B0 CD 21             int 21h
1003 08B2                   _exit: ; выход
1004 08B2 CD 20             int 20h
1005
1006                         ;=== Процедура проверки параметров ком. строки ===;
1007                         ;===
1008 08B4                   commandParamsParser proc
1009 08B4 0E                push CS
1010 08B5 07                pop ES
1011 08B6 C6 06 0180r 00     mov unloadTSR, 0
1012 08BB C6 06 0181r 00     mov notLoadTSR, 0
1013
1014 08C0 BE 0080            mov SI, 80h                      ;SI=смещение
командной строки.
1015 08C3 AC                lodsb                          ;Получим кол-во символов.
1016 08C4 0A C0             or AL, AL                      ;Если 0 символов введено,
1017 08C6 74 40             jz _exitHelp                  ;то все в порядке.
1018
1019 08C8                   _nextChar:
1020
1021 08C8 46                inc SI                          ;Теперь SI указывает на
первый символ +
1022                         строки.
1023
1024 08C9 80 3C 0D           cmp [SI], BYTE ptr 13
1025 08CC 74 3A             je _exitHelp
1026

```

```

1028 08CE AD lodsw ;Получаем два символа
1029 08CF 3D 3F2F cmp AX, '?/' ;Это '/' (данные
расположены в +
1030 обратном порядке, т.е. AL:AH вместо AH:AL)
1031 08D2 74 03 je _question
1032 ;cmp AX, 'u/'
1033 ;je _finishTSR
1034
1035 ;cmp AH, '/'
1036 ;je _errorParam
1037
1038 08D4 EB 32 90 jmp _exitHelp
1039
1040 08D7 _question:
1041 ; вывод строки помощи
1042 08D7 B4 03 mov AH,03
1043 08D9 CD 10 int 10h
1044 08DB BD 0231r lea BP, helpMsg
1045 08DE B9 018D mov CX, helpMsg_length
1046 08E1 B3 07 mov BL, 0111b
1047 08E3 B8 1301 mov AX, 1301h
1048 08E6 CD 10 int 10h
1049 ; конец вывода строки помощи
1050 08E8 F6 16 0181r not notLoadTSR ;флаг того, что
необходимо не загружать резидент
1051 08EC EB DA jmp _nextChar
1052
1053 ;@ === Удаление резидента из памяти ===
1054 ;@ Если по варианту необходимо выгружать резидент по
параметру '/u' командной строки,
1055 ;@ нужно использовать следующий код, в остальных случаях
необходимо закоментировать
1056 ;@ этот код, кроме названия метки! (по желанию можно
избавиться и от метки, но +
1057 аккуратно просмотреть использование)
1058 08EE _finishTSR:
1059 08EE F6 16 0180r not unloadTSR ;флаг того, что
необходимо выгрузить резидент
1060 08F2 EB D4 jmp _nextChar
1061
1062 08F4 EB 12 90 jmp _exitHelp
1063
1064 08F7 _errorParam:
1065 ;вывод строки
1066 08F7 B4 03 mov AH,03
1067 08F9 CD 10 int 10h
1068 08FB BD 03BEr lea BP, CS:errorParamMsg
1069 08FE B9 0025 mov CX, errorParamMsg_length
1070 0901 B3 07 mov BL, 0111b
1071 0903 B8 1301 mov AX, 1301h
1072 0906 CD 10 int 10h
1073 ;конец вывода строки
1074 0908 _exitHelp:
1075 0908 C3 ret
1076 0909 commandParamsParser endp
1077
1078 0909 code ends
1079 end _start

```

Symbol Name	Type	Value
??DATE	Text	"05/13/26"
??FILENAME	Text	"tsr"
??TIME	Text	"14:04:28"
??VERSION	Number	030A

@CPU	Text	0101H
@CURSEG	Text	CODE
@FILENAME	Text	TSR
@WORDSIZE	Text	2
ALREADYINSTALLEDMSG	Byte	CODE:0468
CHANGEFONT	Near	CODE:07F2
CHANGEFX	Near	CODE:04E6
CHARTOCURSIVEINDEX	Byte	CODE:0163
COMMANDPARAMSPARSER	Near	CODE:08B4
COUNTER	Word	CODE:0182
CURSIVEENABLED	Byte	CODE:0152
CURSIVESYMBOL	Byte	CODE:0153
ERRORPARAMMSG	Byte	CODE:03BE
ERRORPARAMMSG_LENGTH	Number	0025
F1_TXT	Byte	CODE:04DE
F2_TXT	Byte	CODE:04E0
F3_TXT	Byte	CODE:04E2
F4_TXT	Byte	CODE:04E4
FX_LENGTH	Number	0002
HELPMMSG	Byte	CODE:0231
HELPMMSG_LENGTH	Number	018D
IGNOREDCHARS	Byte	CODE:0103
IGNOREDLENGTH	Number	0042
IGNOREENABLED	Byte	CODE:0145
INSTALLEDMSG	Byte	CODE:0455
LINE1_LENGTH	Number	0039
LINE2_LENGTH	Number	0039
LINE3_LENGTH	Number	0039
NEW_INT1CH	Far	CODE:063D
NEW_INT2FH	Near	CODE:066B
NEW_INT9H	Far	CODE:0588
NOMEMMSG	Byte	CODE:047E
NOREMOVMSG	Byte	CODE:04C1
NOREMOVMSG_LENGTH	Number	001D
NOTINSTALLEDMSG	Byte	CODE:0492
NOTLOADTSR	Byte	CODE:0181
OLD_INT1CHOFFSET	Word	CODE:0178
OLD_INT1CHSEGMENT	Word	CODE:017A
OLD_INT2FHOFFSET	Word	CODE:017C
OLD_INT2FHSEGMENT	Word	CODE:017E
OLD_INT9HOFFSET	Word	CODE:0174
OLD_INT9HSEGMENT	Word	CODE:0176
PRINTDELAY	Number	0005
PRINTPOS	Word	CODE:0184
PRINTSIGNATURE	Near	CODE:06F8
REMOVEDMSG	Byte	CODE:04B0
REMOVEDMSG_LENGTH	Number	0011
SAVEDSYMBOL	Byte	CODE:0164
SAVEFONT	Near	CODE:07FF
SETCURSIVE	Near	CODE:07A5

SIGNATURELINE1	Byte	CODE:0186
SIGNATURELINE2	Byte	CODE:01BF
SIGNATURELINE3	Byte	CODE:01F8
SIGNATUREPRINTINGENABLED	Byte	CODE:0151
TABLEBOTTOM	Byte	CODE:041C
TABLEBOTTOM_LENGTH	Number	0039
TABLETOP	Byte	CODE:03E3
TABLETOP_LENGTH	Number	0039
TRANSLATEENABLED	Byte	CODE:0150
TRANSLATEFROM	Byte	CODE:0146
TRANSLATELENGTH	Number	0005
TRANSLATETO	Byte	CODE:014B
TRUE	Number	00FF
UNLOADTSR	Byte	CODE:0180
_2FH_EXIT	Near	CODE:06F3

_2FH_STD	Near	CODE:067B
_ACTUALPRINT	Near	CODE:0736
_ALREADYINSTALLED	Near	CODE:08A1
_ALREADY_INSTALLED	Near	CODE:0680
_BLOCK	Near	CODE:060B
_CHECKF5	Near	CODE:04F5
_CHECKF6	Near	CODE:0519
_CHECKF7	Near	CODE:053A
_CHECKF8	Near	CODE:055B
_CHECK_IGNORED	Near	CODE:05FF
_CHECK_TRANSLATE	Near	CODE:0613
_CHECK_TRANSLATE_LOOP	Near	CODE:0620
_DONTPRINT	Near	CODE:0664
_ERRORPARAM	Near	CODE:08F7
_EXIT	Near	CODE:08B2
_EXITHELP	Near	CODE:0908
_EXITSETCURSIVE	Near	CODE:07EF
_EXIT_TMP	Near	CODE:0839
_F1	Near	CODE:05B9
_F2	Near	CODE:05C7
_F8	Near	CODE:059A
_F9	Near	CODE:05A8
_FINISHTSR	Near	CODE:08EE
_GO	Near	CODE:05EF
_GREENF5	Near	CODE:0512
_GREENF6	Near	CODE:0536
_GREENF7	Near	CODE:0557
_GREENF8	Near	CODE:0578
_INITTSR	Near	CODE:080C
_LETSPRINT	Near	CODE:0657
_NEXTCHAR	Near	CODE:08C8
_NOTMEM	Near	CODE:08AB
_NOTREMOVE	Near	CODE:06CE
_NOTREMOVINGNOW	Near	CODE:082F
_NOTTOPRINT	Near	CODE:0669
_OUTFX	Near	CODE:057C
_PRINTBOTTOM	Near	CODE:072F
_PRINTCENTER	Near	CODE:0728
_PRINTTOP	Near	CODE:0721
_QUESTION	Near	CODE:08D7
_QUIT	Near	CODE:0635
_REDF5	Near	CODE:050B

Turbo Assembler Version 3.1
Symbol Table

05/13/26 14:04:29

Page 22

_REDF6	Near	CODE:052F
_REDF7	Near	CODE:0550
_REDF8	Near	CODE:0571
_REMOVE	Near	CODE:0898
_REMOVINGONPARAMETER	Near	CODE:0824
_RESTORESYMBOL	Near	CODE:07E0
_SHIFTABLE	Near	CODE:07B7
_START	Near	CODE:0100
_TEST_FX	Near	CODE:0598
_TMP	Near	CODE:083C
_TRANSLATE	Near	CODE:062C
_TRANSLATE_OR_IGNORE	Near	CODE:05D5
_UNINSTALL	Near	CODE:0683
_UNLOADED	Near	CODE:06E2

Groups & Segments	Bit	Size	Align	Combine	Class
-------------------	-----	------	-------	---------	-------

CODE	16	0909	Para	none	CODE
------	----	------	------	------	------